Complete Reading & Executable R Companion to the catmech Package

How to use this R-package companion

This companion follows Patrick Iben's *Statistical Inference and Two-Phase Audit Design for Contaminated Measurements in Clinical Research: The catmech R Package*, canonical manuscript paper/manuscript.Rmd (`350` lines, SHA-256 `e25e6b44561b4c5b8529d87867eda249665ab2269aa406e8217a90e3639c0758`) and compiled 16-page PDF paper/manuscript.pdf (SHA-256 `79c865d1e4f4ef216599aeb7b4ce931c237bb89b2c525c5b97d50ea6f8b56df1`).

This educational companion explains the statistical definitions, benchmark archives, and R workflows in the frozen catmech manuscript; it does not establish universal interval coverage under unmodeled misspecification, validate per-record error probabilities for individual patient adjudication on diffuse real-world data, or certify FDA 21 CFR Part 11 or ICH E9 regulatory compliance.

Because `catmech` is an executable R package rather than a static paper alone, this companion links four complementary learning modalities:

1. **Complete Reading Companion (`DOCUMENT`):** Work through the thirty source-locked passage units (`C01` through `C30`) below in COMPANION.md or COMPANION.pdf, or take the shorter twelve-lesson route in STUDY_GUIDE.md and STUDY_GUIDE.pdf. COMPLETE_COVERAGE.md maps every line (`1-350`), equation, table, and benchmark archive.
2. **Executable R Lab & Refusal Suite (`run_companion_labs.R` & `interactive_tutor.R`):** Run `Rscript education/run_companion_labs.R` to execute every unit's formula check and four 'When the Package Says No' refusal/warning labs against the installed `catmech` package, or launch `source('education/interactive_tutor.R'); catmech_tutor()` in R for an interactive console walkthrough.
3. **Interactive HTML5 Companion Studio & Browser Simulators (`INTERACTIVE`):** Open companion_lab.html for an offline unit explorer with copy-and-run R cards and the interactive 12-item Transfer & Boundary Practice Quiz (recording attempts, reporting correctness, explaining answers, and supporting retries), alongside the catmech Guide & Method Picker, the Live Linear-MI Audit Simulator, and the Record Audit Planner.
4. **One-Page Visual Decision & Case-Study Cards (`ONE_PAGER`):** Use ONE_PAGER.pdf and the 1-page empirical visual summary catmech-case-study.pdf for rapid clinical trial team reference.

Every unit uses one of six canonical G14 evidence classes: **DEFINITION**, **ANALYTIC**, **MODELED_LOCKED**, **MEASURED_DATED**, **INTERPRETIVE**, and **SCOPE**.

Problem, statistical model, and two-phase audit design

C01. Clinical audit designs, two-phase inference, and the simulation-to-practice synthesis

Where we are: manuscript lines 1-29 (Title, YAML header, structured abstract, and keywords; PDF page 1).

What the passage is doing: The title, metadata, and structured abstract define the scope of the `catmech` R package. In clinical trials, electronic health record (EHR) disease registries, and epidemiological cohorts, continuous measurements are present on nearly all rows but subject to transcription mistakes, unit slips, and assay shifts, while an expensive gold-standard reference measurement is audited on only a fraction of rows (typically 10% to 25%). The abstract summarizes the mathematical contributions (collapsed slice-within-Gibbs mixture posteriors, a rough, unvalidated two-phase audit-size calculation, and Barnard-Rubin small-sample degrees of freedom with CV3, CR1S, and Lin-Wei cluster-robust covariance), the 10 frozen validation archives (over 500,000 requests), the 43% to 48% root mean squared error (RMSE) reduction from retaining recorded values on ClinicalTrials.gov (`N = 27,208`) and NHANES 2017-2018 (`N = 3,908`), and the simulation-to-practice decision rule.

Worked reasoning and R connection: On ClinicalTrials.gov at a 25% audit, discarding unaudited recorded values yields `RMSE = 0.0539`, whereas Simple Linear MI (`catmech_reference(use_recorded = TRUE)`) yields `RMSE = 0.0286`, a `1 - 0.0286 / 0.0539 = 46.9%` reduction. On NHANES, `RMSE` drops from `0.0785` to `0.0409` (`47.9%` reduction). Run `Lab C01` in `education/run_companion_labs.R` or open the interactive Method Picker in `docs/learn/index.html`.

In ordinary language: Audits check a small slice of records against a trusted reference. Keeping recorded numbers cuts error in half compared with throwing unchecked numbers away.

Evidence status: SCOPE of the `catmech` package and MEASURED_DATED summary of the 10 frozen validation archives.

Do not accidentally conclude: Do not conclude that `catmech_audit_trail()` makes an analysis system legally compliant with FDA 21 CFR Part 11 or ICH E9 by itself or that its hashes make the record tamper-evident, or that the contamination mixture model beats simple linear imputation on all real-world datasets.

Try it: On NHANES at a 25% audit, discarding unaudited recorded values gives `RMSE = 0.0785` and Simple Linear MI gives `RMSE = 0.0409`. Verify the percentage RMSE reduction in R with `round(100 * (1 - 0.0409 / 0.0785), 1)`. **Answer:** `47.9%` (matching the abstract's 43% to 48% range across ClinicalTrials.gov and NHANES).

C02. The three-way clinical audit dilemma: naive attenuation, complete-case variance inflation, and discard-recorded loss

Where we are: manuscript lines 30-41 (Section 1 Introduction; PDF page 1-2).

What the passage is doing: Section 1 explains why standard missing-data software does not directly solve the clinical source-data verification (SDV) problem. When an audit uncovers discrepancies between recorded values `W_i` and reference measurements `R_i`, analysts face three flawed conventional choices: (1) naively trusting `W_i`, which attenuates linear regression slopes toward zero and distorts non-linear logistic or Cox models; (2) restricting analysis to the audited subset `A` (complete-case analysis), which discards 75% to 90% of patients and inflates standard errors; or (3) setting unaudited `W_i` values to `NA` and applying standard Missing at Random (MAR) multiple imputation (`mice`, `Amelia`), which throws away the accurate signal contained in uncorrupted recordings.

Worked reasoning and R connection: Under classical additive measurement error `W_i = T_i + e_i` where `Var(T_i) = 4.0` and `Var(e_i) = 1.0`, the ordinary least squares slope is attenuated by the reliability ratio `lambda = Var(T_i) / (Var(T_i) + Var(e_i)) = 4 / (4 + 1) = 0.80` (a 20% bias toward zero). See `Lab C02` in

education/run_companion_labs.R and compare strategies live in docs/learn/simulator.html.

In ordinary language: Analysts have three bad choices after an audit: trust bad numbers, use only checked rows, or erase unchecked rows. Each choice adds bias or wastes data.

Evidence status: DEFINITION of the three conventional audit analysis choices and ANALYTIC errors-in-variables attenuation mechanism.

Do not accidentally conclude: Do not conclude that standard MAR multiple imputation packages (mice, Amelia) are biased when unaudited recorded values are discarded under MAR; their flaw in that mode is precision loss (wider confidence intervals), not bias.

Try it: If true biomarker variance is sigma_T^2 = 4.0 and independent recording error variance is sigma_e^2 = 1.0, what is the classical attenuation factor sigma_T^2 / (sigma_T^2 + sigma_e^2) in R (4 / (4 + 1))? **Answer:** 0.80, meaning a true slope of 1.00 attenuates to 0.80 under naive analysis.

C03. Data generating and contamination framework: latent truth, corruption indicator, recorded value, and Phase II reference

Where we are: manuscript lines 42-56 (Section 2.1, Equations (1)-(3); PDF page 2).

What the passage is doing: Section 2.1 formalizes the two-phase data generating mechanism across N patients in C clinical sites. First, the latent true biological measurement follows a Gaussian linear model T_i | X_i ~ N(X_i beta, sigma_T^2) where X_i includes the analysis outcome and covariates. Second, an unobserved binary contamination indicator delta_i in {0, 1} with stratum-specific probability pi_{s_i} = P(delta_i = 1 | S_i = s_i) (corruption_strata) determines whether the recorded measurement W_i is clean (delta_i = 0, so W_i = T_i) or corrupted (delta_i = 1, so W_i = T_i + epsilon_i with epsilon_i ~ N(mu_e, sigma_e^2) across error_strata). Third, in Phase II, a reference measurement R_i = T_i + eta_i with eta_i ~ N(0, sigma_ref^2) is observed on the audited subset A of size n_audit = |A|, where sigma_ref (reference_sd, defaulting to 0) is a user-declared sensitivity constant for reference assay noise.

Worked reasoning and R connection: When delta_i = 0, (1 - delta_i) T_i + delta_i (T_i + epsilon_i) reduces to W_i = T_i exactly. When delta_i = 1 with T_i = 10.0 and epsilon_i = 3.5, W_i = 13.5. When reference_sd = 0 (the default), every audited measurement R_i is treated as an exact recount R_i = T_i.

In ordinary language: Each row has a hidden true value and a recorded value. A coin flip per stratum decides if a recording stays exact or gets additive noise.

Evidence status: DEFINITION of the two-phase Gaussian contamination mixture model and reference assay noise parameter reference_sd.

Do not accidentally conclude: Do not conclude that catmech_contamination() estimates sigma_ref (reference_sd) from the data; reference_sd is a user-declared sensitivity constant defaulting to 0.

Try it: Using W_i = (1 - delta_i) T_i + delta_i (T_i + epsilon_i) with T_i = 10.0 and epsilon_i = 3.5, evaluate W_i for delta_i = 0 and delta_i = 1 (Lab C03). **Answer:** W_i = 10.0 when delta_i = 0 (clean recording) and W_i = 13.5 when delta_i = 1 (contaminated recording).

C04. Priors and collapsed slice-within-Gibbs MCMC architecture in catmech_contamination

Where we are: manuscript lines 57-60 (Section 2.2 opening paragraph; PDF page 3).

What the passage is doing: Section 2.2 details the Bayesian prior specification and Markov Chain Monte Carlo (MCMC) sampler in `catmech_contamination()`. For exact recounts (`sigma_ref = 0`), the target regression parameters `(beta, sigma_T^2)` receive the noninformative Jeffreys prior `p(beta, sigma_T^2) propto 1 / sigma_T^2` (or a flat prior on `beta` with a scaled inverse-chi-square prior on `sigma_T^2` when `sigma_ref > 0`). Contamination rates receive `pi ~ Beta(a_pi, b_pi)`, and error parameters receive a normal-scaled-inverse-chi-square prior `mu_e | sigma_e^2 ~ N(mu_0, sigma_e^2 / kappa_0)` and `sigma_e^2 ~ Scaled-Inv-chi^2(nu_0, tau_0^2)`. Critically, during MCMC updates of `logit(pi)`, `mu_e`, and `(1/2) log(sigma_e^2)`, the latent variables `(delta_i, T_i)` on unaudited rows are analytically integrated out before applying stepping-out slice sampling (plus a scale-ridge slice step when fewer than 5 discrepancies are observed).

Worked reasoning and R connection: In a standard uncollapsed Gibbs sampler, conditioning on the current imputed `(delta_i, T_i)` creates severe autocorrelation when `pi` or `sigma_e^2` is small. Integrating out `(delta_i, T_i)` analytically collapses the likelihood to a univariate function of `logit(pi)`, `mu_e`, and `(1/2) log(sigma_e^2)` before QR-based conditional draws of `(beta, sigma_T^2)`.

In ordinary language: To keep the chain from sticking, the sampler sums over hidden row values first. Then it updates the error rate, shift, and spread.

Evidence status: ANALYTIC derivation of the collapsed slice-within-Gibbs MCMC update schedule in `catmech_contamination()`.

Do not accidentally conclude: Do not conclude that `catmech_contamination()` runs in C++; `catmech_contamination()` runs collapsed slice-within-Gibbs in R across 4 chains, whereas `catmech_reference()` runs in C++17 via `RcppArmadillo`.

Try it: Why does `catmech_contamination()` integrate out `(delta_i, T_i)` on unaudited rows before slice-sampling `logit(pi)`, `mu_e`, and `(1/2) log(sigma_e^2)`? **Answer:** Collapsing over the unobserved latent indicators and true values reduces serial autocorrelation between the discrete indicators `delta_i` and the global error parameters.

C05. Marginalized corruption probability for unaudited records via Gaussian convolution

Where we are: manuscript lines 61-64 (Section 2.2, Equation (4); PDF page 3).

What the passage is doing: For an unaudited patient (`i not in A`), we observe the recorded measurement `W_i` and covariates `X_i`, but not `T_i`. Under `delta_i = 0` (clean), `W_i = T_i ~ N(X_i beta, sigma_T^2)`. Under `delta_i = 1` (corrupted), `W_i = T_i + epsilon_i` is the sum of two independent Gaussians, yielding the convolved marginal predictive distribution `W_i | delta_i = 1, X_i, Theta ~ N(X_i beta + mu_e, sigma_T^2 + sigma_e^2)`. Applying Bayes' rule yields the exact conditional posterior probability of corruption `p_i = P(delta_i = 1 | W_i, X_i, Theta)` as the ratio of the contaminated Gaussian density `pi * phi(W_i; X_i beta + mu_e, sigma_T^2 + sigma_e^2)` to the two-component mixture density.

Worked reasoning and R connection: Suppose `pi = 0.20`, `X_i beta = 10.0`, `mu_e = 4.0`, `sigma_T^2 = 1.0`, `sigma_e^2 = 3.0`, and we observe `W_i = 14.0`. Under the clean component `N(10, 1)`, `W_i = 14.0` is 4 standard deviations away (`phi = 0.0001338`). Under the contaminated component `N(14, 4)`, `W_i = 14.0` sits at the mean (`phi = 0.19947`). Thus `p_i = (0.20 * 0.19947) / (0.20 * 0.19947 + 0.80 * 0.0001338) = 0.9973` (`Lab C05`).

In ordinary language: For an unchecked row, the model checks two bell curves. It compares the clean curve with the wider error curve.

Evidence status: ANALYTIC closed-form marginal corruption probability `p_i` under Gaussian convolution.

Do not accidentally conclude: Do not conclude that `p_i` has high discrimination when recording errors `epsilon_i` are small relative to `sigma_T`; when `mu_e` and `sigma_e^2` are small compared with `sigma_T^2`, the two Gaussian densities overlap heavily and `p_i` stays near the prior `pi`.

Try it: Run `Lab C05` in R (`pi <- 0.20; w <- 14; num <- pi * dnorm(w, 14, sqrt(1 + 3)); round(num / (num + (1 - pi) * dnorm(w, 10, 1)), 4)`) to compute `p_i` when `W_i = 14.0`. **Answer:** `0.9973`.

C06. Conjugate Gaussian posterior for contaminated unaudited records and reference anchoring

Where we are: manuscript lines 65-70 (Section 2.2, Equations (5)-(6); PDF page 3).

What the passage is doing: Once `delta_i ~ Bernoulli(p_i)` is drawn for an unaudited record (`i not in A`), the latent true value `T_i` is updated conditionally. If `delta_i = 0`, the record is uncorrupted and `T_i = W_i` deterministically. If `delta_i = 1`, `T_i` combines two Gaussian sources of information: the covariate regression prior `T_i | X_i ~ N(X_i beta, sigma_T^2)` and the bias-corrected noisy recording `W_i - mu_e ~ N(T_i, sigma_e^2)`. By Gaussian precision weighting, the conditional posterior is `T_i | delta_i = 1, W_i, X_i, Theta ~ N(mu_{T|W}, sigma_{T|W}^2)` where `sigma_{T|W}^2 = (1 / sigma_T^2 + 1 / sigma_e^2)^(-1)` and `mu_{T|W} = sigma_{T|W}^2 * (X_i beta / sigma_T^2 + (W_i - mu_e) / sigma_e^2)`. For audited records (`i in A`), `R_i` directly anchors `T_i` (`T_i = R_i` when `sigma_ref = 0`), and `m` post-burn-in completions are retained only after passing Gelman-Rubin `R-hat` and effective sample size checks.

Worked reasoning and R connection: Using the numbers from `C05` (`X_i beta = 10.0`, `sigma_T^2 = 1.0`, `W_i = 14.0`, `mu_e = 4.0`, `sigma_e^2 = 3.0`), when `delta_i = 1`, the posterior variance is `(1/1 + 1/3)^(-1) = 0.75` and the posterior mean is `0.75 * (10.0 / 1 + (14.0 - 4.0) / 3) = 10.0` (`Lab C06`).

In ordinary language: If an unchecked row is clean, we keep its number. If it has an error, we blend the model forecast with the shifted number.

Evidence status: ANALYTIC conjugate Gaussian conditional posterior for `T_i` and MCMC convergence gate.

Do not accidentally conclude: Do not conclude that a record with `delta_i = 1` completely discards `W_i`; `W_i - mu_e` still receives precision weight `1 / sigma_e^2` alongside the covariate prediction precision `1 / sigma_T^2`.

Try it: With `sigma_T^2 = 1.0` and `sigma_e^2 = 3.0`, compute the conjugate posterior variance `sigma_{T|W}^2 = (1 / sigma_T^2 + 1 / sigma_e^2)^(-1)` in R (`1 / (1/1 + 1/3)`). **Answer:** `0.75`.

C07. Gaussian reference imputation in C++17: use_recorded = TRUE (Simple Linear MI) versus FALSE

Where we are: manuscript lines 71-72 (Section 2.2 closing paragraph; PDF page 3).

What the passage is doing: When recording discrepancies are subtle and diffuse rather than discrete outliers, positing a binary latent indicator `delta_i` is unnecessary. `catmech_reference()` implements fast conjugate Gaussian reference multiple imputation in C++17 via `RcppArmadillo`. When called with `use_recorded = TRUE` (benchmarked as Simple Linear MI, or `cole_mi`), `catmech_reference()` includes the recorded value `W_i` directly as a continuous linear predictor alongside `X_i` in the audited calibration regression `T_i | W_i, X_i ~ N(gamma_0 + gamma_W W_i + X_i gamma_X, sigma_{T|W,X}^2)`. When called with `use_recorded = FALSE` (the v0.3 default, `reference_normal`), `catmech_reference()` discards `W_i` on unaudited rows and imputes `T_i` solely from `X_i`.

Worked reasoning and R connection: In `docs/learn/simulator.html` (`docs/learn/linear-mi.js`), the browser executes this exact `catmech_reference()` linear calibration model in JavaScript, verified against R across 140 benchmark studies (`docs/learn/qa/compare.js`). In `Lab C16`, switching `use_recorded` from `FALSE` to `TRUE` shrinks the pooled standard error because `W_i` explains substantial variance in `T_i`.

In ordinary language: When errors are small and spread out, we do not need a mixture model. A fast straight-line fit on checked rows adjusts the rest.

Evidence status: DEFINITION of `catmech_reference(use_recorded = TRUE)` (`cole_mi`) versus `catmech_reference(use_recorded = FALSE)` (`reference_normal`).

Do not accidentally conclude: Do not conclude that `catmech_reference()` defaults to `use_recorded = TRUE`; for backward compatibility with v0.3, the formal argument default is `use_recorded = FALSE`, so users should explicitly pass `use_recorded = TRUE` for Simple Linear MI.

Try it: In `catmech_reference()`, what is the difference between `use_recorded = TRUE` and `use_recorded = FALSE` on unaudited rows? **Answer:** `use_recorded = TRUE` retains `W_i` as a continuous linear calibration predictor alongside `X_i`, whereas `use_recorded = FALSE` discards `W_i` on unaudited rows and imputes from `X_i` alone.

Two-phase audit planning, multi-endpoint pooling, and calibration

C08. Two-phase influence function variance decomposition and the interval-width precision ratio

Where we are: manuscript lines 73-80 (Section 2.3, Equations (7)-(8); PDF page 3-4).

What the passage is doing: Section 2.3 derives the relationship between the Phase II audit fraction `f = n_audit / N` and downstream confidence interval width using semiparametric influence function theory (Neyman 1938; Cochran 1977; Breslow and Cain 2003; Angelopoulos et al. 2023). Let `psi_i` denote the efficient influence function of the target regression parameter `theta` and `G_i = (W_i, Y_i, X_i)` denote the Phase I variables observed on all `N` patients. The asymptotic variance at audit fraction `f` decomposes into the full-cohort census variance `(1 /`

N) Var(psi_i) plus the Phase II incomplete-audit penalty ((1 - f) / (f N)) Var(psi_i - E[psi_i | G_i]). Taking the square root of the ratio of Var(theta_hat_f) to Var(theta_hat_1) yields the confidence interval width precision ratio rho(f) = Width(f) / Width(1) = sqrt(1 + k (1 - f) / f), where k = Var(psi_i - E[psi_i | G_i]) / Var(psi_i) measures the fraction of influence-function variance left unexplained by Phase I data.

Worked reasoning and R connection: When k = 0.27 and we audit f = 0.25 (25% of the cohort), (1 - f) / f = 3, so rho(0.25) = sqrt(1 + 0.27 * 3) = sqrt(1.81) = 1.3454 (meaning the confidence interval is 34.54% wider than a 100% census audit). See Lab C08 and explore docs/audit-planner/index.html.

In ordinary language: Checking only part of a study widens the error bars by a square-root factor. That factor depends on the checked share and leftover noise.

Evidence status: ANALYTIC two-phase asymptotic variance decomposition and precision ratio rho(f).

Do not accidentally conclude: Do not conclude that k is known without error before or during a small pilot audit; uncertainty in k_hat is the primary reason empirical audit-size planning is noisy.

Try it: If k = 0.27 and the audit fraction is f = 0.25, compute the precision ratio rho(0.25) = sqrt(1 + k * (1 - f) / f) in R (round(sqrt(1 + 0.27 * 3), 4)). **Answer:** 1.3454.

C09. Rough, unvalidated audit-size planning (catmech_audit_power) and the pre-specified NO-GO boundary on catmech_audit_plan

Where we are: manuscript lines 81-84 (Section 2.3, Equation (9) and validation disclosure; PDF page 4).

What the passage is doing: Because the noise parameter k is estimated with uncertainty, catmech_audit_power() is a rough, unvalidated planning aid. For each candidate audit fraction it reports the share of plausible k values whose precision ratio meets a target, Share(n_audit) = P(sqrt(1 + k_hat (1 - f) / f) <= rho_target), across n_sim = 2000 log-normal Monte Carlo draws of k_hat. The draws come from assumed error_rate and correlation values or from the bias-corrected and accelerated (BCa) bootstrap bounds of a pilot fitted with catmech_audit_plan(). The output labels this share power, but it describes uncertainty about k; it is not the power of a hypothesis test. Without a pilot plan, the default k assumes the unexplained variation sits in erroneous records that the analysis can identify, which is optimistic, so the function also reports the audit size that linear imputation needs to meet the target under the double-sampling regression estimator (Cochran 1977) at the same correlation. No study has checked these recommendations. The manuscript also discloses that in pre-specified validation testing (validation/audit-plan-study, 14,400 target fits and 7,200 plans), catmech_audit_plan() received a NO-GO verdict because its median error in predicting interval width at a 5% audit fraction was 11.3% to 16.8% (exceeding the 10% tolerance).

Worked reasoning and R connection: With the manuscript's workflow settings (N = 1000, error_rate = 0.20, correlation = 0.85, and the default r_squared = 0.30), the default k is about 0.040 and the function recommends a 10% audit (n = 100). Linear imputation at the same correlation has k = (1 - 0.85^2) * (1 - 0.30) = 0.194 and needs about 25.7% (n = 257) to reach a width ratio of 1.25, roughly two and a half times as many records (Lab C09). catmech_audit_plan() stays an exploratory calculation rather than a validated sample-size guarantee.

In ordinary language: The audit-size tool makes a rough guess from assumed error rates, and nobody has checked its answers. The pilot planner also missed its ten percent goal.

Evidence status: ANALYTIC definition of the rough audit-size share (`catmech_audit_power()`, not validated) and MEASURED_DATED `NO-GO` validation verdict on `catmech_audit_plan()`.

Do not accidentally conclude: Do not read the `power` column of `catmech_audit_power()` as the power of a hypothesis test or its recommendation as a validated audit size, and do not treat `catmech_audit_plan()` as a certified sample-size calculator; its pre-specified validation verdict is `NO-GO` (`11.3%` to `16.8%` median width error at `f = 5%`).

Try it: Run `catmech_audit_power(N = 1000, target_ratio = 1.25, power_target = 0.80, error_rate = 0.20, correlation = 0.85, fractions = seq(0.10, 0.50, by = 0.05))` and compare its recommendation with the linear-imputation line of the printout (`Lab C09`). **Answer:** It recommends `n = 100` records (`10%`), while linear imputation at the same correlation needs about `n = 257` (`25.7%`, `k = 0.194`); both are rough guides that no study has checked.

C10. Rubin multiple imputation combining rules and the fraction of missing information

Where we are: manuscript lines 85-90 (Section 2.4, Equations (10)-(11); PDF page 4).

What the passage is doing: Section 2.4 presents Rubin's (1987) combining rules across `j = 1, ..., m` completed datasets. Given point estimates `theta_hat_j` and within-imputation covariance matrices `U_hat_j`, the pooled point estimate is the average `theta_bar = (1/m) sum theta_hat_j`, the within-imputation variance is `U_bar = (1/m) sum U_hat_j`, and the between-imputation variance is the sample covariance `B = (1/(m - 1)) sum (theta_hat_j - theta_bar)(theta_hat_j - theta_bar)^T`. The total covariance matrix is `T = U_bar + (1 + 1/m) B`, where the `(1 + 1/m)` factor accounts for using a finite number of imputations `m`. The diagonal fraction of missing information (FMI) is `lambda_hat = (1 + 1/m) diag(B) / diag(T)`.

Worked reasoning and R connection: Suppose `m = 5`, within-imputation variance is `U_bar = 0.04`, and between-imputation variance is `B = 0.01`. Then `(1 + 1/m) B = 1.2 * 0.01 = 0.012`, total variance is `T = 0.04 + 0.012 = 0.052`, and `lambda_hat = 0.012 / 0.052 = 0.2308` (`Lab C10-C11`).

In ordinary language: Across filled-in datasets, we add within-set spread to between-set spread. A small boost adjusts for using a finite number of sets.

Evidence status: ANALYTIC Rubin (1987) multiple imputation combining rules for point estimates, total variance `T`, and fraction of missing information `lambda_hat`.

Do not accidentally conclude: Do not conclude that total variance is simply `U_bar + B`; omitting the `1 / m` finite-imputation correction underestimates total variance when `m` is small.

Try it: For `m = 5`, `U_bar = 0.04`, and `B = 0.01`, compute the total variance `T = U_bar + (1 + 1/m) B` and `lambda_hat` in R (`Lab C10-C11`). **Answer:** `T = 0.052` and `lambda_hat = 0.012 / 0.052 = 0.2308`.

C11. Barnard and Rubin small-sample degrees of freedom adjustment for finite samples, clusters, and events

Where we are: manuscript lines 91-94 (Section 2.4, Equation (12); PDF page 4).

What the passage is doing: Classical Rubin degrees of freedom `nu_m = (m - 1) lambda_hat^(-2)` assume that the complete-data degrees of freedom `nu_com` are infinite. When `m` is large and `lambda_hat` is small, `nu_m` can exceed the total number of patients `N`, clinical sites `C`, or survival events `d`, producing anti-conservative (too narrow) confidence intervals. `catmech` implements the Barnard and Rubin (1999) finite-sample adjustment: `nu_obs = ((nu_com + 1) / (nu_com + 3)) * nu_com * (1 - lambda_hat)` and `nu_BR = (nu_m * nu_obs) / (nu_m + nu_obs)`.

Worked reasoning and R connection: Because `nu_BR` is half the harmonic mean of `nu_m` and `nu_obs`, it is strictly bounded below both `nu_m` and `nu_obs < nu_com`. In `Lab C10-C11`, with `m = 5`, `lambda_hat = 0.2308`, and `C = 10` clusters (`nu_com = C - 1 = 9`), classical `nu_m = 4 / (0.2308^2) = 75.11`, which is more than 8 times the number of clusters! The Barnard-Rubin adjustment yields `nu_obs = (10/12) * 9 * (1 - 0.2308) = 5.769` and `nu_BR = 5.3577`, protecting finite-cluster coverage.

In ordinary language: Old formulas can claim more degrees of freedom than the study has sites or events. The Barnard-Rubin formula caps degrees of freedom below the complete-data limit.

Evidence status: ANALYTIC Barnard and Rubin (1999) finite-sample degrees of freedom formula `nu_BR`.

Do not accidentally conclude: Do not conclude that increasing the number of imputations `m` to infinity lets `nu_BR` grow without bound; as `m -> infinity`, `nu_m -> infinity` and `nu_BR` converges to `nu_obs < nu_com`.

Try it: In `Lab C10-C11`, with `m = 5`, `lambda_hat = 0.2308`, and `nu_com = 9` (`C = 10` sites), compare `nu_m` with `nu_BR`. **Answer:** Classical `nu_m = 75.11` (exceeding `nu_com = 9`), whereas `nu_BR = 5.3577` remains strictly below `nu_com = 9`.

C12. Downstream pooling for continuous (CV3 jackknife) and generalized linear (CR1S sandwich) endpoints

Where we are: manuscript lines 95-98 (Section 2.4, Endpoint items 1 and 2, Equation (13); PDF page 4-5).

What the passage is doing: Section 2.4 specifies how `catmech` computes within-imputation covariance `U_hat_j` and complete-data degrees of freedom `nu_com` for continuous and generalized linear endpoints. (1) In `catmech_pool_lm()`, unclustered models use ordinary least squares covariance with `nu_com = N - p`; clustered models across `C` clinical sites compute the closed-form delete-one-cluster jackknife (`cv3`) covariance `((C - 1) / C) sum_{c=1}^C (beta_hat_{(-c)} - beta_hat)(beta_hat_{(-c)} - beta_hat)^T` and pool with `nu_com = C - 1`. (2) In `catmech_pool_glm()` (logistic and Poisson regression), unclustered models use inverse Fisher information with `nu_com = N - p`; clustered models set `nu_com = C - 1` and compute the CR1S finite-sample-corrected Huber-White-Liang-Zeger cluster-robust sandwich covariance `V_hat_sandwich = (C / (C - 1)) * ((N - 1) / (N - p)) * J_hat^(-1) (sum_{c=1}^C s_c s_c^T) J_hat^(-1)`, with optional exponentiation (`exponentiate = TRUE`) for odds ratios or incidence rate ratios.

Worked reasoning and R connection: When C is small (e.g., 8 to 20 clinical sites), uncorrected cluster sandwich estimators underestimate variance. Both cv3 (for linear models) and the C/(C-1) * (N-1)/(N-p) factor with nu_com = C - 1 (for GLMs) guard against small-cluster overconfidence (Lab C17-C18).

In ordinary language: Linear models use a leave-one-site-out jackknife variance across clinical sites. Binary and count models use a small-sample site sandwich.

Evidence status: ANALYTIC within-imputation covariance estimators and complete-data degrees of freedom for catmech_pool_lm() (cv3) and catmech_pool_glm() (CR1S).

Do not accidentally conclude: Do not conclude that clustered GLM pooling uses nu_com = N - p; when cluster is supplied across C clinical sites, nu_com is set to C - 1.

Try it: In a study with N = 240 patients across C = 8 clinical sites and p = 4 regression parameters, what is nu_com in catmech_pool_lm(..., cluster = 'site')? **Answer:** nu_com = C - 1 = 7 (not N - p = 236).

C13. Time-to-event Cox proportional hazards pooling with event-based degrees of freedom and Lin-Wei sandwich

Where we are: manuscript lines 99-100 (Section 2.4, Endpoint item 3; PDF page 5).

What the passage is doing: Item 3 of Section 2.4 defines catmech_pool_coxph() for right-censored time-to-event survival endpoints via survival::coxph(). When cluster = NULL (unclustered), within-imputation covariance is the inverse partial Fisher information matrix and complete-data degrees of freedom are defined strictly as nu_com = d - p, where d is the number of observed failure events (not total patients N) and p is the parameter count. When cluster is specified across C clinical sites, catmech_pool_coxph() computes the cluster-summed Lin and Wei (1989) score-residual sandwich covariance and sets nu_com = C - 1. Hazard ratios HR = exp(theta_bar) and confidence intervals are formed on the log-hazard scale using the Barnard-Rubin t-distribution with nu_BR degrees of freedom before exponentiating.

Worked reasoning and R connection: In a rare-event oncology trial with N = 1,000 patients but only d = 25 events and p = 3 covariates, partial likelihood information comes from the 25 events, so nu_com = 25 - 3 = 22 rather than 997. When clustered across C = 8 sites (Lab C17-C18), nu_com = 8 - 1 = 7 and all(pool_cox_res$df < 7.0).

In ordinary language: In time-to-event models, real facts come from observed events, not censored rows. So degrees of freedom count events minus model terms.

Evidence status: ANALYTIC event-based degrees of freedom (d - p) and clustered Lin-Wei sandwich pooling in catmech_pool_coxph().

Do not accidentally conclude: Do not conclude that catmech_pool_coxph() forms confidence intervals by adding +/- 1.96 * SE directly to the hazard ratio; intervals are computed on the log-hazard scale using t_{nu_BR} critical values and then exponentiated.

Try it: In an unclustered Cox model with N = 500 patients, d = 40 observed events, and p = 4 covariates, what is nu_com in catmech_pool_coxph()? **Answer:** d - p = 40 - 4 = 36.

C14. Out-of-sample cross-validated predictive calibration checking via rank PIT and its slope-bias limit

Where we are: manuscript lines 101-106 (Section 2.5, Equation (14); PDF page 5).

What the passage is doing: Section 2.5 introduces `catmech_holdout_check()`, which partitions audited records into `K` stratified (or cluster-grouped) cross-validation folds, masks the reference values `R_i` in fold `k`, and refits the imputation model on the remaining `K - 1` folds. Because the contamination mixture predictive distribution has a discrete point mass at `W_i` (retained with probability `1 - p_i`) and uses `m` finite draws, standard continuous PIT values would pile up at ties. Equation (14) therefore computes the tie-randomized empirical rank Probability Integral Transform (PIT) `u_i = (sum I(T_{i,-k}^{(j)} < R_i^*) + U_i (sum I(T_{i,-k}^{(j)} = R_i^*) + 1)) / (m + 1)` with `U_i ~ Uniform(0, 1)`. It flags `miscalibration_detected = TRUE` if a two-sided Kolmogorov-Smirnov test of `u_i` against `Uniform(0, 1)` gives `p < 0.01`, if the 95% Wilson upper bound on empirical predictive coverage falls below nominal, or if the corruption-rate `z`-statistic exceeds `z_{0.995}`.

Worked reasoning and R connection: If all `m = 20` held-out draws equal `R_i^*` exactly (a clean recording tie), `sum I(<) = 0` and `sum I(=) = 20`, so `u_i = U_i * (20 + 1) / 21 = U_i ~ Uniform(0, 1)`, preventing false KS alarms on exact ties! However, as Section 5.1 shows, row-level predictive checks have low power against regression slope bias.

In ordinary language: Holdout checking hides part of the audit and tests if predictions match the hidden truth. Tie-breaking keeps clean matches uniform, but passing does not prove slope accuracy.

Evidence status: ANALYTIC tie-randomized empirical rank PIT formula and SCOPE boundary of `catmech_holdout_check()`.

Do not accidentally conclude: Do not conclude that `miscalibration_detected = FALSE` in `catmech_holdout_check()` proves that downstream regression slopes are unbiased; observation-level predictive checks had `0/40` alarm power in `nonlinear_target` and `1/40` in `heavy_tail_target`.

Try it: In Equation (14), if `m = 20` and all `20` held-out predictive draws equal `R_i^*` (`sum I(<) = 0`, `sum I(=) = 20`), what does `u_i` simplify to? **Answer:** `u_i = (0 + U_i * (20 + 1)) / (20 + 1) = U_i ~ Uniform(0, 1)`.

Software architecture and illustrative 8-stage clinical workflow

C15. Modular S3 software architecture across planning, diagnostics, imputation, pooling, and provenance

Where we are: manuscript lines 107-118 (Section 3 opening and module table; PDF page 5-6).

What the passage is doing: Section 3 maps the `catmech` package into five functional S3 modules: (1) **Audit planning** (`catmech_audit_plan()`, exploratory, and `catmech_audit_power()`, a rough, unvalidated aid), (2) **Data diagnostics** (`catmech_recording_audit()`, `catmech_holdout_check()`), (3) **Imputation engines** (`catmech_contamination()`, `catmech_reference()`), (4) **Model pooling** (`catmech_pool_lm()`, `catmech_pool_glm()`, `catmech_pool_coxph()`, `catmech_compare_strategies()`), and (5) **Provenance** (`catmech_sensitivity()`, `catmech_audit_trail()`).

Worked reasoning and R connection: Every module communicates through self-describing S3 objects (`catmech_result`, `catmech_pooled_lm`, `catmech_pooled_glm`, `catmech_pooled_coxph`, `catmech_recording_audit`, `catmech_holdout_check`, `catmech_audit_trail`) carrying SHA-256 assumption hashes and convergence diagnostics. In `Refusal Lab 1` (`education/run_companion_labs.R`), setting

`fit$model_diagnostics$convergence_pass <- FALSE` causes `catmech_pool_lm()` to immediately refuse pooling rather than silently returning invalid intervals.

In ordinary language: The package has five parts: planning, checks, filling in data, pooling, and hash logs. Each step checks the prior step before running.

Evidence status: DEFINITION of the five modular S3 subsystems in `catmech`.

Do not accidentally conclude: Do not conclude that the pooling functions (`catmech_pool_lm()`, `catmech_pool_glm()`, `catmech_pool_coxph()`) accept arbitrary raw data frames; they require a verified `catmech_result` object (and refuse to pool if `convergence_pass = FALSE`).

Try it: Run `Lab C15` in `education/run_companion_labs.R` (`catmech_recording_audit(cohort, 'lab_value', 'lab_ref', by = 'site', agreement_tol = 0.5)`). Which S3 class is returned? **Answer:** `catmech_recording_audit` (containing overall and per-site Jeffreys beta discrepancy intervals).

C16. Comparison with mice, simex, mecor, and Amelia, plus broom generics and strategy comparison

Where we are: manuscript lines 119-133 (Section 3 capability comparison table and closing paragraph; PDF page 6).

What the passage is doing: Section 3 compares `catmech` against four established CRAN packages: `mice` (van Buuren and Groothuis-Oudshoorn 2011), `simex` (Lederer and Kuchenhoff 2006), `mecor` (Nab et al. 2021), and `Amelia` (Honaker et al. 2011). While `mice` and `Amelia` provide general-purpose MAR multiple imputation and `mecor` provides linear regression calibration, none combine two-phase audit-size planning (rough and unvalidated in `catmech`), reference-calibrated contamination mixture modeling, multi-endpoint pooling (linear with CV3, GLM with CR1S sandwich, and Cox PH with event-based `d - p` df and Lin-Wei sandwich), rank-PIT holdout screening, and SHA-256 provenance trails. Section 3 also highlights `tidy()` and `glance()` S3 generics for tidyverse workflows and `catmech_compare_strategies()` for side-by-side comparison of `validation_subsample`, `naive_trust_recorded`, `linear_mi_recorded`, and `contamination_model`.

Worked reasoning and R connection: Rather than forcing users to trust one model, `catmech_compare_strategies()` runs all four audit strategies on the same dataset and plots their point estimates and confidence intervals side by side (`Lab C26`).

In ordinary language: Other R tools handle missing rows or straight-line noise on their own. None join audit sizing, mixture fits, site pooling, and hash logs.

Evidence status: DEFINITION of software capabilities across `catmech`, `mice`, `simex`, `mecor`, and `Amelia`.

Do not accidentally conclude: Do not conclude that `catmech` replaces `mice` for general multivariate missing-data patterns across arbitrary columns; `catmech` is purpose-built for two-phase reference-calibrated measurement verification and downstream pooling.

Try it: Which four benchmark strategies does `catmech_compare_strategies()` evaluate side by side (`Lab C26`)? **Answer:** `validation_subsample` (complete-case), `naive_trust_recorded` (naive analysis), `linear_mi_recorded`

(`catmech_reference(use_recorded = TRUE)`), and `contamination_model` (`catmech_contamination()`).

C17. Reproducible 8-stage clinical trial workflow (Stages 1-5): simulation, audit sizing, screening, mixture fit, and calibration

Where we are: manuscript lines 134-173 (Section 4 code chunk (Stages 1-5); PDF page 6-7).

What the passage is doing: Section 4 demonstrates the first five stages of a reproducible 10-site clinical trial workflow (`N = 1,000`, 25% reference audit `n_audit = 250`): (1) simulating the cohort via `simulate_catmech_lab(n = 1000, seed = 42)` with site IDs, binary adverse events, right-censored survival times, and the Nelson-Aalen cumulative hazard from `mice::nelsonaalen(cohort, time, event)`; (2) a rough, unvalidated audit-size calculation via `catmech_audit_power(N = 1000, target_ratio = 1.25, power_target = 0.80, error_rate = 0.20, correlation = 0.85)`; (3) site-stratified discrepancy screening via `catmech_recording_audit(..., by = 'site', agreement_tol = 0.5)`; (4) reference-calibrated contamination mixture imputation via `catmech_contamination(..., predictors = c('age', 'sex', 'inflammation', 'outcome', 'adverse_event', 'event', 'hazard'), corruption_strata = 'outlier_flag', m = 10)`; and (5) 5-fold out-of-sample predictive calibration checking via `catmech_holdout_check(fit = fit, folds = 5L, m = 20L)`.

Worked reasoning and R connection: The lab value is a predictor in all three Stage 6 analyses, so `predictors` in Stage 4 includes every analysis outcome: `outcome` for the linear model, `adverse_event` for the logistic model, and, for the Cox model, the event indicator `event` and the Nelson-Aalen cumulative hazard `hazard` (White and Royston 2009). Leaving an outcome out biases the lab value's coefficient toward zero. Under `catmech` 1.0.1, `catmech_pool_glm()` and `catmech_pool_coxph()` warn when the outcome or event indicator is missing from the imputation model; with all four included, the workflow runs without warnings (`Lab C17-C18`).

In ordinary language: Steps one to five build a ten-site study and make a rough guess at audit size. Then they check site error rates, fit the model with every outcome included, and test held-out rows.

Evidence status: DEFINITION of the reproducible R workflow for Stages 1-5 of a clinical audit analysis.

Do not accidentally conclude: Do not leave any downstream analysis outcome out of `predictors` when calling `catmech_contamination()` or `catmech_reference()` for a value that later serves as a predictor: include a continuous or binary outcome itself, and for a Cox model the event indicator and the Nelson-Aalen cumulative hazard. Omitting them biases the downstream coefficient toward zero (a hazard ratio toward 1).

Try it: Why do `adverse_event`, `event`, and `hazard` appear in `predictors` in Stage 4 when imputing `lab_value`? **Answer:** `lab_value` is a predictor in the logistic and Cox models of Stage 6, so the imputation model must condition on each analysis outcome; for the Cox model that means the event indicator and the Nelson-Aalen cumulative hazard (White and Royston 2009). Leaving them out biases the lab value's log odds ratio and log hazard ratio toward zero, and `catmech_pool_glm()` and `catmech_pool_coxph()` warn.

C18. Reproducible 8-stage clinical trial workflow (Stages 6-8): multi-endpoint pooling, strategy comparison, and provenance

Where we are: manuscript lines 174-210 (Section 4 code chunk (Stages 6-8) and console output; PDF page 7-8).

What the passage is doing: Stages 6-8 complete the clinical workflow: (6) pooling continuous (`catmech_pool_lm` with `cluster = 'site'`), binary (`catmech_pool_glm` with `family = binomial(link = 'logit')`, `cluster = 'site'`, `exponentiate = TRUE`), and survival (`catmech_pool_coxph` with `Surv(time, event) ~ lab_value + age + sex`, `cluster = 'site'`, `exponentiate = TRUE`) models; (7) comparing all four audit strategies via `catmech_compare_strategies()`; and (8) generating a Markdown provenance record with SHA-256 hashes of the data, the specification, and the completions via `catmech_audit_trail()`. The printed console output for `pool_survival` (`m = 10`, `n = 1000`, `Events: 735`) shows `lab_value` log-hazard `estimate = 0.1877`, clustered Lin-Wei `std.error = 0.0386`, Barnard-Rubin `df = 6.0881` (bounded below `nu_com = C - 1 = 9`), and exponentiated hazard ratio `estimate_exp = 1.2065` (95% CI `[1.0982, 1.3255]`). The simulation in Stage 1 draws survival times with log hazard ratio `0.25` per unit of the true lab value, and the interval `[0.0936, 0.2818]` contains it.

Worked reasoning and R connection: Verify that `exp(0.1877) = 1.2065` and that with `df = 6.0881`, the critical `t`-multiplier is `qt(0.975, df = 6.0881) = 2.4384` (wider than `1.96`), giving `0.1877 +/- 2.4384 * 0.0386 = [0.0936, 0.2818]`. Exponentiating those rounded limits gives `[1.0981, 1.3255]`, which matches the printed `[1.0982, 1.3255]` up to rounding of the limits.

In ordinary language: Steps six to eight pool three model types across ten sites. Then they compare all four methods and write the hash log.

Evidence status: MODELED_LOCKED illustrative workflow output and ANALYTIC Barnard-Rubin `t`-interval exponentiation.

Do not accidentally conclude: Do not conclude that `df = 6.0881` in the 1,000-patient, 735-event Cox model is a bug; because `cluster = 'site'` was specified across `C = 10` sites, `nu_com = C - 1 = 9`, so `df` is properly bounded below 9.

Try it: From the printed Cox PH table (`estimate = 0.1877`, `std.error = 0.0386`, `df = 6.0881`), verify `estimate_exp` in R (`round(exp(0.1877), 4)`). **Answer:** `1.2065`.

Synthetic simulation studies and clustered variance benchmarks

C19. Design of the 52,950-request synthetic contamination study across 27 scenarios and 7 strategies

Where we are: manuscript lines 211-218 (Section 5 opening and Section 5.1 introduction; PDF page 8).

What the passage is doing: Section 5 introduces the pre-specified Monte Carlo validation program spanning 10 frozen archives and over 500,000 replicate requests. The primary reference-calibrated contamination study (`validation/contamination-study`) executed `52,950` planned requests across `183` method-scenario cells: 4 primary cells (`clean`, `flagged_errors`, `missed_flags`, `site_shift_declared`, `1,000` cohorts each) and 23 stress cells (`150` cohorts each), with cohort size `N = 600`, true linear regression slope `beta = 0.800` on `T_i`, a 25% reference audit (`n_audit = 150`), and `m = 20` imputations. Across all `52,950` requests,

52,900 intervals were returned alongside 16 expected sparse-stratum design refusals (14 in reference_05pct, 2 in reference_10pct), 33 MCMC convergence refusals, and 1 singular raking fit.

Worked reasoning and R connection: Every single one of those 52,950 requests (52,900 + 16 + 33 + 1 = 52,950) is accounted for and independently re-verified by python3 tools/verify-artifacts.py and python3 paper/verify_paper_numbers.py.

In ordinary language: The test suite ran 52,950 planned fits across four main settings and 23 stress tests. It logged every result and every refusal.

Evidence status: MODELED_LOCKED pre-specified simulation study design (validation/contamination-study, 52,950 requests).

Do not accidentally conclude: Do not conclude that non-convergent MCMC fits or sparse-stratum refusals were silently dropped or replaced with new random seeds; all 52,950 planned requests (52,900 intervals + 50 refusals/errors) are explicitly tallied.

Try it: Verify that the returned intervals and refusals in validation/contamination-study sum to the exact planned total in R (52900 + 16 + 33 + 1). **Answer:** 52,950.

C20. Synthetic benchmark results in missed_flags and error_gross_and_small, plus the four primary cells

Where we are: manuscript lines 219-240 (Section 5.1 benchmark table and primary cell analysis; PDF page 9).

What the passage is doing: Section 5.1 compares the seven core strategies on missed_flags (1,000 cohorts, unflagged errors) and error_gross_and_small (150 cohorts, mixing 10x gross transcription outliers with small noise). In missed_flags, Flag Masking (v02_mask_normal) suffers severe attenuation bias (-0.0815, 55.9% coverage), while Complete-Case (gold_only, width 0.3309, RMSE 0.0814) and Discard Recorded MI (reference_normal, width 0.2769, RMSE 0.0684) are unbiased but wide. The Contamination Mixture model achieves bias -0.0048, 94.9% coverage, width 0.1842 (narrowing CIs by 33.5% vs reference_normal and 44.3% vs gold_only), and RMSE 0.0449. In error_gross_and_small, gross outliers break Simple Linear MI (bias -0.2354, 64.7% coverage, RMSE 0.3964) and Generalized Raking (bias -0.2844, 53.7% coverage, RMSE 0.4451), whereas catmech_contamination() maintains 94.0% coverage, bias -0.0117, and RMSE 0.0447. Across the four 1,000-cohort primary cells, catmech_contamination() passed the [0.925, 0.975] Wilson coverage screen in three cells (96.5%, 96.3%, 94.9%) and narrowly missed in site_shift_declared (93.7%, Wilson interval [0.9202, 0.9505]).

Worked reasoning and R connection: Why does Simple Linear MI (cole_mi) collapse in error_gross_and_small? Because a few 10x transcription outliers pull the OLS calibration slope gamma_W toward zero, distorting imputations for all clean unaudited rows! The mixture model isolates those outliers via delta_i = 1.

In ordinary language: When big or unflagged errors appear in test data, the mixture model stays on target. It cuts bar width by 44% over using checked rows alone.

Evidence status: MODELED_LOCKED synthetic benchmark results across missed_flags, error_gross_and_small, and the four primary 1,000-cohort cells.

Do not accidentally conclude: Do not conclude that `catmech_contamination()` passed the strict Wilson interval screen in all four primary cells; in `site_shift_declared`, point coverage was `93.7%` and its Wilson lower bound `0.9202` fell slightly below `0.925`.

Try it: From the `missed_flags` table, verify the percentage reduction in mean CI width of Contamination Mixture (`0.1842`) relative to Complete-Case (`gold_only`, `0.3309`) in R (`round(100 * (1 - 0.1842 / 0.3309), 1)`). **Answer:** `44.3%`.

C21. Pre-specified stress-cell failure boundaries and holdout calibration alarm power

Where we are: manuscript lines 241-249 (Section 5.1 stress-cell failure inventory; PDF page 9-10).

What the passage is doing: Under the pre-specified stress failure threshold (coverage `< 90.0%` or `|bias| > 0.030`), `catmech_contamination()` failed in three of the 23 stress cells: (1) `heavy_tail_target` (Student-`t_4` target `T_i`, bias `+0.0330`, `87.3%` coverage, because genuine tail extremes in `T_i` are misclassified as recording errors and shrunk toward `X_i beta`, whereas `reference_normal` [`93.3%`], `cole_mi` [`92.0%`], and `raking` [`92.0%`] held coverage); (2) `nonignorable_missed` (audit selection depending directly on latent `T_i` with unflagged errors, bias `-0.0506`, `79.3%` coverage, whereas design-consistent `raking` achieved bias `+0.0070` and `87.3%` coverage); and (3) `noisy_undeclared` (reference noise `sigma_ref > 0` left at `reference_sd = 0`, bias `-0.0261`, `88.7%` coverage, repaired to `-0.0071` bias and `91.8%` coverage in `noisy_declared` when `reference_sd` is declared). In addition, `catmech_holdout_check()` had `0/40` false alarms in `clean` and `missed_flags`, `18/40` (45%) alarms in `site_shift_undeclared`, `1/40` in `heavy_tail_target`, and `0/40` in `nonlinear_target`.

Worked reasoning and R connection: In `Refusal Lab 3` (`education/run_companion_labs.R`), leaving `reference_sd = 0` when `R_i` has assay noise causes `catmech_contamination()` to emit a loud warning that nearly all audited rows disagree!

In ordinary language: Stress tests show three traps for the mixture model. It struggles with heavy tails, biased audit choice, and unstated reference noise.

Evidence status: MODELED_LOCKED stress-cell failure boundaries and `catmech_holdout_check()` alarm power evaluation.

Do not accidentally conclude: Do not conclude that `catmech_contamination()` is robust to heavy-tailed target distributions (`t_4`); when `T_i` has heavy tails, `cole_mi` (`92.0%` coverage) or `reference_normal` (`93.3%` coverage) should be used instead.

Try it: What happens in `noisy_undeclared` (`88.7%` coverage) when the user declares the known reference assay noise via `reference_sd` (`noisy_declared`)? **Answer:** Bias improves from `-0.0261` to `-0.0071` and coverage recovers to `91.8%` (compared with `52.7%` for `reference_normal` and `38.0%` for `cole_mi`).

C22. Clustered variance study (148,500 requests): why CV3 beats CR1S/CR2 and the predictor-attenuation boundary

Where we are: manuscript lines 250-258 (Section 5.2; PDF page 10).

What the passage is doing: Section 5.2 reports the `148,500`-request clustered variance and multilevel imputation benchmark (`148,486` returned intervals and `14` refusals) across simulated clustered cohorts and facility recounts of 11 indicators from the Rwanda Health Management

Information System (HMIS). Standard small-sample sandwich (`cr1s`) and Bell-McCaffrey / Pustejovsky-Tipton (`cr2`, which inflated widths by up to 95% under random-intercept MI and passed only 23 of 44 simulated cells) failed pre-specified eligibility. The delete-one-cluster jackknife (`cv3`) restored complete-data recount coverage (`gold_only`) to `92.0%-97.5%` across all 11 Rwanda indicators and passed eligibility under random-intercept outcome imputation (`>= 92.0%` on 9 of 11 Rwanda indicators and passing 36 of 44 simulated cells). However, two boundaries emerged: (1) `cv3` did not rescue unclustered linear MI on Rwanda HMIS outcomes (coverage `< 92%` on 7 of 11 indicators, verdict `PARTIAL`); and (2) when the audited variable `T_i` is a **predictor** (`Y ~ T + x + stratum`), `catmech_reference(cluster = )` attenuated slopes by up to `-16.1%` (`84.4%` coverage at 25% audit), whereas fitting `catmech_reference(use_recorded = TRUE)` without `cluster` and pooling with `catmech_pool_lm(cluster = )` achieved `-3.0%` bias and `95.4%` coverage.

Worked reasoning and R connection: `Refusal Lab 2` in `education/run_companion_labs.R` triggers the exact warning emitted by `catmech_pool_lm()` when a user passes a `catmech_reference(cluster = )` fit as a predictor!

In ordinary language: Across sites, the leave-one-site-out jackknife gives the best bar coverage. Yet site-level random intercepts shrink slopes when the checked variable is a predictor.

Evidence status: MEASURED_DATED and MODELED_LOCKED clustered variance benchmark (`148,500` requests) and `PARTIAL` accuracy verdict.

Do not accidentally conclude: Do not use `catmech_reference(cluster = )` when the audited variable is a predictor in the downstream regression; fit `catmech_reference(use_recorded = TRUE)` without `cluster` and pool with `catmech_pool_lm(cluster = )`.

Try it: In the 148,500-request clustered study, what bias and coverage did `catmech_reference(use_recorded = TRUE)` (without `cluster`) pooled with `catmech_pool_lm(cluster = )` achieve when `T_i` was a predictor at a 25% audit? **Answer:** `-3.0%` bias and `95.4%` coverage (versus `-16.1%` bias and `84.4%` coverage for `catmech_reference(cluster = )`).

Real-world empirical benchmarks and methodological decision framework

C23. Public paired-discrepancy benchmarks on ClinicalTrials.gov and NHANES (33,800 requests)

Where we are: manuscript lines 259-277 (Section 6.1 and empirical benchmark table; PDF page 11).

What the passage is doing: Section 6.1 evaluates `catmech` across `33,800` planned requests (`33,613` returned intervals, `76` design refusals, `83` MCMC convergence refusals, `28` singular raking errors) on two large public datasets with real paired discrepancies on every row: ClinicalTrials.gov (`N = 27,208` studies, `14.2%` discrepancy rate, target slope `0.0357` log months / log N) and NHANES 2017-2018 (`N = 3,908` adults, `67.3%` discrepancy rate `> 1` kg, target slope `0.127` mmHg / kg). At a 25% audit (`n_audit = 150` of `N = 600`), retaining recorded values cut RMSE by `43%` to `48%` and narrowed CIs by `48%` compared with discarding unaudited values (`Discard Rec.`, RMSE `0.0539` on CT.gov and `0.0785` on NHANES). On ClinicalTrials.gov, where naive bias was tiny (`+0.0043`), `Linear MI` achieved bias `+0.0001` and RMSE `0.0286` (beating `Contamination`

at +0.0068 and 0.0308, and avoiding Contamination's 18% refusal rate at a 5% audit). On NHANES, where self-reported weight had substantial naive bias (-0.0180, 14% of target slope), both Linear MI (bias -0.0025, RMSE 0.0409) and Contamination (bias -0.0066, RMSE 0.0408) corrected the bias.

Worked reasoning and R connection: Notice that on ClinicalTrials.gov, even the Full-Reference Oracle covered 91.2% because homoskedastic OLS standard errors (0.0244) slightly underestimated empirical sampling SD (0.0275).

In ordinary language: On 27,208 trial records and 3,908 health exams, keeping recorded numbers helps a lot. It cuts error by 43% to 48% over dropping unchecked rows.

Evidence status: MEASURED_DATED empirical benchmark across ClinicalTrials.gov (N = 27,208) and NHANES 2017-2018 (N = 3,908; 33,800 requests).

Do not accidentally conclude: Do not conclude that ClinicalTrials.gov or NHANES is a patient-level clinical trial chart re-abstraction (SDV) dataset, or that catmech_contamination() outperforms Linear MI on ClinicalTrials.gov (Linear MI had lower bias +0.0001 vs +0.0068 and lower RMSE 0.0286 vs 0.0308).

Try it: On NHANES at a 25% audit (target slope 0.127 mmHg / kg), how did Simple Linear MI (use_recorded = TRUE) change regression bias compared with Naive (Trust Rec.)?
Answer: It reduced bias from -0.0180 (14% of the target slope) to -0.0025 while holding 93.9% coverage.

C24. Why per-record corruption probabilities fail on diffuse real-world discrepancies (AUC 0.543 and 0.500)

Where we are: manuscript lines 278-279 (Section 6.1 closing paragraph; PDF page 11).

What the passage is doing: Section 6.1 highlights a critical empirical finding: although per-record posterior corruption probabilities p_i = P(delta_i = 1 | W_i, X_i, Theta) achieved AUC = 0.962 for identifying corrupted unaudited rows in the synthetic missed_flags cell (where errors were large relative to sigma_T), on real-world data p_i had **near-chance discrimination**: AUC = 0.543 on ClinicalTrials.gov (Brier score 0.127 vs 0.121 for base-rate prevalence alone; posterior mean RMSE 0.277 vs 0.252 for raw recorded values) and AUC = 0.500 on NHANES (Brier score 0.222 vs 0.220 for prevalence; posterior mean RMSE 4.81 vs 4.73 kg for raw recorded values). Why? Because real-world discrepancies in both datasets were small relative to the natural residual spread of T_i | X_i. While the KS alarm in catmech_holdout_check() stayed quiet (2/38 on CT.gov, 1/40 on NHANES), the continuous held-out Brier and RMSE skill scores correctly showed no predictive edge over raw recorded values.

Worked reasoning and R connection: This is why catmech explicitly forbids using p_i to flag or adjudicate individual patient records on diffuse real-world data (Lab S10 in education/interactive_tutor.R).

In ordinary language: On real data with small errors, the mixture model cannot tell which single rows are wrong. Its row scores do no better than a coin flip.

Evidence status: MEASURED_DATED empirical ROC-AUC and Brier skill evaluation of per-record corruption probabilities p_i.

Do not accidentally conclude: Do not use individual record-level corruption probabilities `p_i` from `catmech_contamination()` to adjudicate or discard specific patient rows in diffuse real-world audits (`AUC = 0.543` on ClinicalTrials.gov and `0.500` on NHANES).

Try it: Compare the ROC-AUC of per-record corruption probabilities `p_i` in synthetic `missed_flags` versus ClinicalTrials.gov and NHANES (`Lab S10`). **Answer:** `AUC = 0.962` in synthetic `missed_flags`, dropping to near-chance `0.543` on ClinicalTrials.gov and `0.500` on NHANES.

C25. Real-audit slope recovery benchmark (45,600 requests) on Rwanda HMIS and psychology recounts

Where we are: manuscript lines 280-289 (Section 6.2; PDF page 12).

What the passage is doing: Section 6.2 reports the pre-specified `45,600`-request real-audit slope recovery study (`validation/REAL_AUDIT_SYNTHESIS.md`) on Rwanda HMIS facility register recounts and `498` psychology meta-analysis effect-size recounts under three decision rules at a 25% audit: (1) **Rule 1 (`Linear MI` vs raw recorded)**: `Linear MI` paid clearly on **1 of 4 Rwanda slopes** (`ANC 4th visits`, where errors scaled with facility size `r = +0.477` and recovery RMSE fell to `0.458x` raw recorded), tied on `PNC` (`0.923x` on 500 cohorts, `1.009x` on 2,000 fresh seeds), and had higher RMSE than raw recorded on `Syphilis` (`2.393x`) and `Iron` (`1.125x`). (2) **Rule 2 (`catmech_contamination()` vs `Linear MI`)**: Verdict **`INCONCLUSIVE`**. Although `catmech_contamination()` had lower point RMSE on `PNC` (`0.053` vs `0.072`, ratio `0.745`) and `Iron` (`0.112` vs `0.156`, ratio `0.717`), it failed the coverage condition on all three primary slopes (`71.6%` on `PNC`, `66.6%` on `Syphilis`, `35.4%` on `Iron`, and `49.7%` on `ANC 4th visits`) and failed the `>= 90%` return rate on two (`80.8%` on `Syphilis`, `84.8%` on `Iron`), with per-record error AUCs of `0.501` to `0.582`. (3) **Rule 3 (Psychology controls)**: Where raw reported slopes already matched truth, `Linear MI` at 25% inflated RMSE by `2.705x` (SMD) and `1.578x` (r), whereas `catmech_contamination()` stayed near recorded values (`1.306x` and `0.950x`).

Worked reasoning and R connection: Together, Rules 1-3 show that `catmech_contamination()`'s interval advantage remains **synthetic-only** (under gross, separable errors), while `Linear MI` has much safer coverage (`>= 79.5%`) on moderate real-world recounts.

In ordinary language: On Rwanda clinic recounts, the mixture model had lower point error on two slopes. Yet its bars missed the truth too often, so neither model won clearly.

Evidence status: MEASURED_DATED real-audit regression slope recovery benchmark (`45,600` requests) across Rwanda HMIS and psychology recounts.

Do not accidentally conclude: Do not claim that `catmech_contamination()` beat Simple Linear MI overall on Rwanda HMIS; the pre-specified Rule 2 verdict is `INCONCLUSIVE` because clustered 95% coverage fell to `35.4%-71.6%`.

Try it: In the Rwanda HMIS real-audit study at a 25% audit, what was the clustered 95% interval coverage of `catmech_contamination()` vs `Linear MI` on `PNC` and `Iron`? **Answer:** On `PNC`, `71.6%` vs `85.4%` (`95.2%` for recount oracle); on `Iron`, `35.4%` vs `79.5%`.

C26. Methodological decision framework: choosing between Simple Linear MI and Contamination Mixture

Where we are: manuscript lines 290-314 (Section 7 decision tree and rules; PDF page 12-13).

What the passage is doing: Section 7 synthesizes the synthetic and empirical evidence into an operational two-step decision protocol: first screen discrepancy rates and stratum heterogeneity with `catmech_recording_audit()` and compare candidate strategies on audited rows with `catmech_compare_strategies()`. Then branch on the error regime: (1) **Gross, separable contamination** (e.g., decimal slips or 10x unit errors with Gaussian target residuals and ignorable audit selection): use `catmech_contamination()`, verifying `>= 90%` return rate and positive holdout Brier/RMSE skill over raw recorded values; (2) **Subtle, diffuse, or moderate real-world discrepancies** (rounding, mild reporting bias, moderate recount diffs, or heavy-tailed targets): use Simple Linear MI (`catmech_reference(use_recorded = TRUE)`), and when the audited variable is a predictor in a clustered study, fit `catmech_reference(use_recorded = TRUE)` without `cluster` and pool with `catmech_pool_lm(cluster = )`.

Worked reasoning and R connection: You can walk through all 24 combinations of study design, error structure, and clustering interactively in the Method Picker at `docs/learn/index.html` (which generates the exact R snippet for each branch, verified in `tools/verify-artifacts.py`).

In ordinary language: Use the mixture model only when errors are huge outliers and true noise is bell-shaped. For everyday moderate errors, use simple straight-line filling.

Evidence status: INTERPRETIVE methodological decision rule grounded in the synthetic (`Section 5`) and empirical (`Section 6`) benchmarks.

Do not accidentally conclude: Do not use `catmech_contamination()` as a blind default without checking return rates (`>= 90%`) and held-out Brier/RMSE skill scores against raw recorded values.

Try it: According to the Section 7 decision tree, which function and argument should be used when recording discrepancies are diffuse or moderate in a clustered study where the audited variable is a predictor? **Answer:** Fit `catmech_reference(use_recorded = TRUE)` without `cluster`, and pool with `catmech_pool_lm(cluster = )`.

Discussion, regulatory provenance, verification, and availability

C27. Discussion: simulation-to-practice divergence and why catmech centers on a diagnostic workflow

Where we are: manuscript lines 315-328 (Section 8 Discussion; PDF page 13-14).

What the passage is doing: Section 8 reflects on the central scientific lesson of the paper: the divergence between synthetic Monte Carlo simulations and real-world empirical audits. In synthetic benchmarks where contamination is bimodal, gross, and drawn from the exact parametric family assumed by the estimator (`missed_flags` and `error_gross_and_small`), `catmech_contamination()` eliminates linear-MI attenuation bias (`-0.0117` vs `-0.2354`) and narrows CIs by `44.3%` over complete-case analysis (`0.1842` vs `0.3309`). Yet even synthetic stress tests expose hard boundaries (`heavy_tail_target` `87.3%`, `nonignorable_missed` `79.3%`, `noisy_undeclared` `88.7%`), and across ClinicalTrials.gov, NHANES, and Rwanda HMIS, real discrepancies are diffuse and non-separable (`p_i` AUC `0.500` to `0.582`), making `catmech_reference(use_recorded = TRUE)` the safer practical baseline.

Worked reasoning and R connection: This divergence explains why `catmech` ships as a three-part diagnostic toolkit (`catmech_recording_audit()`, `catmech_holdout_check()`, and `catmech_compare_strategies()`) rather than a black-box single estimator.

In ordinary language: Big fake outliers favor mixture models, but real audit errors are often small and spread out. That is why the package checks and compares methods first.

Evidence status: INTERPRETIVE synthesis of simulation-to-practice divergence and diagnostic workflow design.

Do not accidentally conclude: Do not rely on the Kolmogorov-Smirnov uniformity alarm in `catmech_holdout_check()` alone to justify the mixture model; compare continuous held-out Brier and RMSE skill scores against raw recorded values.

Try it: What are the three diagnostic functions that Section 8 recommends running to evaluate whether a dataset supports mixture modeling, linear MI, complete-case analysis, or raw recorded values? **Answer:** `catmech_recording_audit()`, `catmech_holdout_check()`, and `catmech_compare_strategies()`.

C28. Regulatory documentation and data governance: what catmech_audit_trail records and where its scope stops

Where we are: manuscript lines 329-337 (Section 9 Regulatory documentation and data governance; PDF page 14).

What the passage is doing: Section 9 defines the exact role of `catmech_audit_trail()` in regulated clinical trial workflows. FDA 21 CFR Part 11 governs electronic records and audit trails, and ICH E9 asks that statistical data handling and imputation methods be pre-specified and documented. `catmech_audit_trail()` supports that documentation by emitting a structured provenance report in Markdown, JSON, or standalone HTML containing: (1) execution metadata (R version, OS platform, `catmech` version, UTC timestamp); (2) SHA-256 hashes of the input data frame and of each of the `m` completed datasets, computed over the serialized R data frames (so they can be recomputed only in R, not with a file-hashing tool), and of the assumption specification as canonical JSON text; (3) MCMC convergence status (`convergence_passed` and `mcmc_iterations`, with full `R-hat` and bulk/tail ESS vectors in `result$model_diagnostics`); and (4) pooled regression tables and sensitivity envelopes (`catmech_sensitivity()`). A rerun can be compared with the hashes, but they do not cover the pooled results and do not make the record tamper-evident, because anyone who edits the data can also regenerate the record.

Worked reasoning and R connection: As enforced in `R/audit-trail.R` (`Lab C28`), the report is titled `Catmech Audit Trail & Provenance Report`, and its footer `### Audit Trail & Provenance Record` says the hashes do not cover the pooled results or make the document tamper-evident. Hashing a dataset supports reproducibility and documentation, but regulatory compliance depends on the surrounding validated system, access controls, and standard operating procedures.

In ordinary language: The trail tool logs SHA-256 hashes of the data and settings, plus chain checks. A rerun can be compared with it, but it cannot prove the data were untouched or grant legal approval.

Evidence status: DEFINITION and SCOPE of `catmech_audit_trail()` provenance reports.

Do not accidentally conclude: Do not claim that running `catmech_audit_trail()` certifies FDA 21 CFR Part 11 or ICH E9 compliance or makes the record tamper-evident; compliance is a property of the institutional system, access controls, and procedures around the analysis, and anyone who edits the data can regenerate the record.

Try it: Which three sets of SHA-256 hashes does `catmech_audit_trail()` record inside its `integrity` block (`Lab C28`), and which one is taken over canonical JSON text rather than a serialized R data frame? **Answer:** The original input data hash (`original_data_sha256`), the assumption specification hash (`assumption_specification_sha256`), and the list of `m` completed dataset hashes (`completed_datasets_sha256`); the specification hash is of canonical JSON text, and the data and completion hashes are over serialized R data frames.

C29. Quality assurance and verification: 1,812 test expectations, R CMD check, and 500,000+ replicate checks

Where we are: manuscript lines 338-345 (Section 10 Quality assurance and verification; PDF page 14).

What the passage is doing: Section 10 summarizes the three-layer verification harness backing `catmech` 1.0.1: (1) `199` `testthat` tests with `1,812` expectations across `19` test files, passing with `0` failures, `0` warnings, and `0` skips; (2) `R CMD check --as-cran` reporting `0` errors and `0` warnings across macOS (`aarch64-apple-darwin23`), Ubuntu Linux (`x86_64-pc-linux-gnu`), and Windows Server (`x86_64-w64-mingw32`); and (3) independent Python verification scripts (`tools/verify-artifacts.py`, `tools/verify-release-archive.py`, `paper/verify_paper_numbers.py`, and `docs/case-study/fact_check.py`) that recompute summary statistics, paired Monte Carlo standard errors, source hashes, and negative controls (such as planted bias mutations and deleted scenarios) across all 10 frozen benchmark archives and over `500,000` requests.

Worked reasoning and R connection: Every claim in this educational companion is additionally verified by `education/verify_educational_companion.py`, `Rscript education/run_companion_labs.R`, and `g14_companion_gate.py`.

In ordinary language: The package passes 1,812 test checks and CRAN checks with zero errors or warnings. Python scripts also re-check more than 500,000 benchmark fits.

Evidence status: MEASURED_DATED software quality assurance and artifact verification inventory.

Do not accidentally conclude: Do not conclude that passing `R CMD check --as-cran` and `verify-artifacts.py` establishes universal statistical validity on unmodeled data; as `verify-artifacts.py` prints on its final line, mechanical verification proves correspondence and repeatability, not inferential validity.

Try it: How many `testthat` expectations does `catmech` 1.0.1 pass, and what does `R CMD check --as-cran` report for errors and warnings? **Answer:** `1,812` expectations in `199` tests (`0` failures, `0` warnings, `0` skips) and `0` errors and `0` warnings under `R CMD check --as-cran`.

C30. Software availability, MIT license, R/C++17 dependencies, and bibliographic references

Where we are: manuscript lines 346-350 (Section 11 Availability and Section 12 References; PDF page 14-16).

What the passage is doing: Sections 11 and 12 record software licensing, runtime dependencies, repository availability, and the 25 bibliographic references in `paper/paper.bib`. `catmech` is released under the permissive MIT license and requires `R (>= 4.1.0)` and a `C++17` compiler. Its non-base CRAN imports are `Rcpp`, `RcppArmadillo`, `digest`, `jsonlite`, `mice`, `pan`, `posterior`, `survival`, `truncnorm`, and `yaml` (alongside base R packages `graphics`,

grDevices, parallel, stats, and utils). The references span classical two-phase sampling (Neyman 1938; Cochran 1977; Robins et al. 1994; Breslow and Cain 2003; Angelopoulos et al. 2023), multiple imputation (Rubin 1987, 1996; Barnard and Rubin 1999; van Buuren and Groothuis-Oudshoorn 2011; White et al. 2011; Honaker et al. 2011; Huang et al. 2013), measurement error (Carroll et al. 2006; Lederer and Kuchenhoff 2006; Nab et al. 2021), cluster-robust inference (Liang and Zeger 1986; Lin and Wei 1989), predictive calibration and MCMC (Dawid 1984; Gelman and Rubin 1992), regulatory guidance (ICH E9 1998; FDA Part 11 2003), and empirical audit datasets (NHANES 2018; Nshimyiryo et al. 2020; Maassen et al. 2020).

Worked reasoning and R connection: You can inspect the exact package imports at any time in R via `packageDescription('catmech', fields = c('Version', 'License', 'Imports'))`.

In ordinary language: The code is free under the MIT license for R 4.1 or newer with C++17. It draws on 25 papers on sampling, missing data, and audits.

Evidence status: DEFINITION of software license, CRAN package dependencies, and bibliographic foundation (`paper/paper.bib`).

Do not accidentally conclude: Do not conclude that `catmech` requires proprietary external solvers or internet access at runtime; all C++17 and R engines execute locally using standard CRAN packages.

Try it: How many non-base CRAN packages does `catmech` import in `DESCRIPTION`? **Answer:** `10` non-base CRAN packages (`Rcpp`, `RcppArmadillo`, `digest`, `jsonlite`, `mice`, `pan`, `posterior`, `survival`, `truncnorm`, and `yaml`), plus 5 base R packages.

Glossary of statistical and software terms

- **Audited reference (`R_i`):** Gold-standard measurement obtained in Phase II on a subset `A` of size `n_audit` via chart re-abstraction, recount, or central lab re-assay.
- **Barnard-Rubin degrees of freedom (`nu_BR`):** Finite-sample degrees of freedom adjustment (`Barnard & Rubin 1999`) that combines classical MI degrees of freedom `nu_m` with complete-data degrees of freedom `nu_com` (`N - p`, `C - 1`, or `d - p`) so `nu_BR < nu_com`.
- **Collapsed slice-within-Gibbs:** MCMC sampler in `catmech_contamination()` that analytically integrates out unobserved `(delta_i, T_i)` on unaudited rows before slice-sampling `logit(pi)`, `mu_e`, and `(1/2) log(sigma_e^2)`.
- **CV3 delete-one-cluster jackknife:** Cluster-robust variance estimator `((C - 1) / C) sum (beta_hat_{(-c)} - beta_hat)(beta_hat_{(-c)} - beta_hat)^T` used in `catmech_pool_lm(cluster = )`.
- **CR1S sandwich covariance:** Small-sample-adjusted `(C/(C-1) * (N-1)/(N-p))` Huber-White-Liang-Zeger cluster-robust sandwich covariance used in `catmech_pool_glm(cluster = )`.
- **Lin-Wei sandwich covariance:** Cluster-summed score-residual sandwich covariance (`Lin & Wei 1989`) used in `catmech_pool_coxph(cluster = )`.
- **Rank PIT (`u_i`):** Tie-randomized empirical Probability Integral Transform computed by `catmech_holdout_check()` across K-fold cross-validation on audited rows.
- **Simple Linear MI (`cole_mi`):** Calling `catmech_reference(..., use_recorded = TRUE)`, which includes the recorded value `W_i` as a continuous linear predictor alongside covariates `X_i` in C++17 (`RcppArmadillo`).

Locked 12-item transfer assessment (4 CORE, 4 BOUNDARY, 4 TRANSFER)

- **T01 (CORE):** Why does discarding unaudited recorded values W_i and applying standard MAR multiple imputation (reference_normal) lose precision compared with catmech_reference(use_recorded = TRUE)?
  Answer key: Because most unaudited recorded values W_i contain substantial valid signal about T_i; retaining W_i as a calibration predictor cuts RMSE by 43% to 48% on ClinicalTrials.gov and NHANES.
- **T02 (CORE):** In catmech_contamination(), how is the latent true value T_i updated on an unaudited row when delta_i = 0 versus delta_i = 1?
  Answer key: When delta_i = 0 (uncorrupted), T_i = W_i deterministically; when delta_i = 1 (corrupted), T_i is drawn from the precision-weighted conjugate Gaussian posterior combining X_i beta and W_i - mu_e.
- **T03 (CORE):** How does catmech prevent anti-conservative confidence intervals when pooling across m imputations in studies with few clinical sites C or few survival events d?
  Answer key: It applies the Barnard-Rubin (1999) finite-sample adjustment nu_BR with nu_com = C - 1 for clustered models (using CV3, CR1S, or Lin-Wei covariance) and nu_com = d - p for unclustered Cox PH models.
- **T04 (CORE):** Why did Simple Linear MI (cole_mi) break down in the synthetic error_gross_and_small stress scenario while catmech_contamination() maintained 94.0% coverage?
  Answer key: Gross 10x transcription outliers severely attenuated the linear calibration slope gamma_W (bias -0.2354, 64.7% coverage), whereas the contamination mixture isolated gross outliers via delta_i = 1.
- **T05 (BOUNDARY):** Does a passing result (miscalibration_detected = FALSE) from catmech_holdout_check() prove that downstream regression slopes are unbiased?
  Answer key: No; held-out row checks test observation-level prediction and had low power against regression slope bias (0/40 alarms in nonlinear_target and 1/40 in heavy_tail_target).
- **T06 (BOUNDARY):** Can per-record posterior corruption probabilities p_i from catmech_contamination() be used to flag or adjudicate individual patient rows in diffuse real-world audits?
  Answer key: No; when discrepancies are diffuse relative to T_i | X_i, per-record p_i has near-chance discrimination (AUC = 0.543 on ClinicalTrials.gov, 0.500 on NHANES, and 0.501-0.582 on Rwanda HMIS).
- **T07 (BOUNDARY):** What is the validation status of catmech_audit_plan(), and why?
  Answer key: It carries a pre-specified NO-GO validation status (exploratory only) because its median error in predicting interval width at a 5% audit fraction was 11.3% to 16.8%, exceeding the 10% tolerance.
- **T08 (BOUNDARY):** Does generating a report with catmech_audit_trail() certify that an analysis is compliant with FDA 21 CFR Part 11 and ICH E9?
  Answer key: No; SHA-256 provenance records support reproducibility and regulatory documentation, but compliance depends on the surrounding validated system, access controls, and standard operating procedures.
- **T09 (TRANSFER):** A multicenter trial has C = 12 sites and audits a continuous biomarker T_i on 25% of patients to estimate a linear regression Y ~ T + age + sex. Should the analyst fit catmech_reference(cluster = 'site')?
  Answer key: No; because the audited variable T_i is a predictor rather than the outcome, clustered random-intercept MI attenuates the slope (up to -16.1%). Fit catmech_reference(use_recorded = TRUE) without cluster and pool with catmech_pool_lm(cluster = 'site').

- **T10 (TRANSFER):** In a biomarker audit where the true physiological distribution T_i is known to have heavy Student-t_4 tails and discrepancies are moderate, which catmech engine should be chosen?
  Answer key: Choose catmech_reference(use_recorded = TRUE) (Simple Linear MI, 92.0% coverage in heavy_tail_target) rather than catmech_contamination(), which misclassifies genuine tail extremes as recording errors (87.3% coverage).
- **T11 (TRANSFER):** When auditing a noisy reference assay where gold-standard measurements R_i have known analytical standard deviation $\sigma_{ref} = 0.35$, what happens if reference_sd is left at 0 versus set to 0.35?
  Answer key: Leaving reference_sd = 0 triggers a universal-disagreement warning and undercoverage (88.7% in noisy_undeclared); setting reference_sd = 0.35 (noisy_declared) restores bias (-0.0071) and coverage (91.8%).
- **T12 (TRANSFER):** An unclustered oncology survival study has $N = 800$ patients, $d = 30$ observed deaths, and $p = 4$ covariates in catmech_pool_coxph(). What is the upper bound on the Barnard-Rubin degrees of freedom ν_{BR}?
  Answer key: Because unclustered Cox PH pooling defines complete-data degrees of freedom strictly from observed failure events as $\nu_{com} = d - p = 30 - 4 = 26$, ν_{BR} is strictly less than 26.